

ПЛАНОВЕ ПО ИНФОРМАТИКА НА IX КЛАС

Тема 1: Езици и среди за програмиране

1. Разлика между писане на код и програмиране? – алгоритмите

- * програмиране – (if, else, for...) C, C++, Java, Python...
- * писане на код – (ключови думи <>) HTML, CSS, SQL...
- * блендиране (смесване) на 2 езика като C# + HTML = .cshtml

Синтаксис – правилата

Семантика – смисъл

2. Видове езици за програмиране според нивата на абстракция:

Ниво	Език	Превод	Характеристики	Приложения
Ниско	Машинен, Асемблер	Интерпретатор	Максимална скорост и контрол, много сложни – с буквени и цифрени съкращения	Драйвери, firmware, системно програмиране
Средно	C, C++, Rust	Компилятор	Висока скорост, баланс между контрол и абстракция	Операционни системи, игри, софтуер с висока производителност
Високо	Python, Java, C#	Виртуална машина / Интерпретатор	Лесни за учене, по-бавни	Уеб приложения, data science, AI
Много високо	SQL, MATLAB	Специализиран локален домейн	Не са универсални	Заявки към бази данни, математически и научни изчисления

Транслаторът - проверява за синтактични грешки и след отстраняването им превежда програма, написана на език от високо ниво на машинен език. Бива 2 вида – компилатор и интерпретатор.

- **Компилатор:** превежда целия програмен код на машинен език преди изпълнение, създавайки отделен изпълним файл. Компилираните програми са по-бързи, но грешките се откриват при компилация. Езици C++, Java...
- **Интерпретатор:** превежда и изпълнява кода ред по ред по време на работа, без да създава отделен изпълним файл. Интерпретираните програми са по-бавни, но грешките се откриват по време на изпълнение. Езици Python, PHP, JS
- **Виртуална машина (Java):** Това е "хибридният" подход. Кодът се компилира до междинен bytecode, който се изпълнява от виртуална машина (JVM). Това прави езика преносим "write once, run anywhere".

3. Важни дати:

- **1843 г** – първата програма от Ада Лъвлейс (първият програмист в историята) – създава алгоритъм за изчисляване на числата на Бернули за аналитичната машина на Чарлз Бабидж.
- **1993 г** – HTML – HyperText Markup Language – език за маркиране на текст с ХИПЕРлинкове за навигация от един документ в друг. Тим Бърнърс-Лий – създава първия сървър и интернет.

ООП – Обобщение

ООП = Обектно-ориентирано програмиране. Обекти от реалния заобикалящ свят се представят в електронен вид въз основа на свои характеристики.

Разлика между информация и данни:

Информация – в нашето пространство / околна среда

Данни – когато тази информация се вкара в компютъра

Какво е клас? – Шаблон за нови обекти от сложен-съставен тип (съставен е от различни прости типове данни – int, string, double и т.н.).

Как описваме обектите? – Като НДУ – необходимо и достатъчно условие.

Клас = шаблон

```
public class Student
{
    public string Name { get; set; }
    public int Age { get; set; }
}
```

Обект = инстанция

```
Student student1 = new Student { Name = "Иван", Age = 17 };
```

Модификатори за достъп

public, private, protected, internal + 2 смесени

- public -> достъп отвсякъде
- private -> само в класа
- protected -> клас + наследници
- internal -> същия проект

Кой модификатор е по подразбиране, ако не е изписан? – internal

internal class Program който пази входната точка на програмата – Main()

Може ли класът да бъде private? – не.

Какви са характеристиките на обектите?

- 1) **Public properties** (с главна буква, достъпни навсякъде / видими в Main());
- 2) **Private fields** (с малка буква)

```
class Student
{
    public string Name { get; set; };
    public int Age { get; set; }

    public void SayHello()
```

```
{
    Console.WriteLine("Здравей, аз съм " + Name);
}
```

```
Student s1 = new Student();
s1.Name = "Иван";
s1.Age = 17;
s1.SayHello();
```

Има ли конструктор в примера? – Не или празния по подразбиране
 Конструктор stor + Tab + Tab – Конструира нов обект – Специален метод **()**, който се извиква веднага при създаване на нов обект, носи името на класа и няма тип за връщана стойност (без void, int, double). Може да бъде с параметри или без (по подразбиране). Може да имаме и няколко конструктора в един клас (полиморфизъм / презареждане на имена). Наследяване на конструктори? – По подразбиране е само празният, а останалите с **: base**

```
public class GameCharacter
{
    public string Name { get; }
    public int Health { get; }
    public int Level { get; }

    public GameCharacter(string name, int health, int level) // ГЛАВЕН конструктор
    {
        Name = name;
        Health = health;
        Level = level;
        Console.WriteLine($"Създаден герой: {name}, HP: {health}, Level: {level}");
    }

    public GameCharacter(string name, int health) // извиква главния и го разширява с 1 променлива
        : this(name, health, 1)
    {
        Console.WriteLine("Използван конструктор с 2 параметъра");
    }

    public GameCharacter(string name)
        : this(name, 100, 1) // извиква главния и го разширява с 2 променливи
    {
        Console.WriteLine("Използван конструктор с 1 параметър");
    }

    public GameCharacter() // Конструктор по подразбиране (без параметри)
```

```
        : this("Hero", 100, 1) // извиква главния с всички стойности
    {
        Console.WriteLine("Използван конструктор по подразбиране");
    }
}
```

: this се използва за "**constructor chaining**" - верижно извикване на конструктори от същия клас.

```
class BankAccount
{
    private decimal balance;

    public decimal Balance
    {
        get { return balance; }
    }

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            balance += amount;
    }
}
```

в **set** използваме ключова дума **value** за проверките.

4-те принципа на ООП:

- 1) **Капсулация** - слоев обхват на видимост **{}**
- 2) **Наследяване** - преизползване на код чрез **:**
- 3) **Полиморфизъм** - едно име на метод с много реализации заради различни input variables
- 4) **Абстракция** - отдалечаване (опростяване, скриване на сложността)

Капсулация - скриване на данни и поведение – за да стигнем до защитен private, ни е необходим междинен видим public като негова обвивка (балон).

```
public class BankAccount
{
    private double balance; // скрита данна
```

```

public void Deposit(double amount) // видим метод
{
    if (amount > 0) balance += amount;
}

public double GetBalance() => balance; // достъпване чрез междинен обвиващ видим метод
}

```

Наследяване - преизползване на код

```

public class Animal { } // РОДИТЕЛ

public class Cat : Animal { } // наследник
public class Dog : Animal { }
public class Bird : Animal, IFlyable { } // наследява точно 1 РОДИТЕЛ + 1 (или много) интерфейси

```

Полиморфизъм - "много форми"

```

public class Animal { public virtual void MakeSound() { } }

public class Dog : Animal { public override void MakeSound() => Console.WriteLine("Bark!"); }

```

Абстракция - скриване на сложността

```

public abstract class Shape
{
    public abstract double CalculateArea();
}

```

Как да различаваме virtual от abstract методи в РОДИТЕЛСКИТЕ КЛАСОВЕ без да ги бъркаме?

- **virtual** има тяло {} – предназначен е за примерно реализиране, което може да се използва по желание, а може и да се презапише чрез override. В примера тялото е празно {} но го има.

- **abstract** – отдалечаване, опростяване – няма тяло ; и задължително трябва да му го дадем. Не може да се извиква име на метод, без да му се даде решение.

Синтаксиси за **валидация**:

- * автоматични свойства prop + Tab + Tab които са съкратени само до { get; set; }

get = read-only

set = write-only

* за допълнителна валидация използваме по-стария и подробен синтаксис. При него се качваме над декларациите и добавяме private fields, които след това може да използваме в public properties.

```
public class Product
{
    private string id;
    private decimal price;

    public string Id
    {
        get { return id; }
        set { id = value; }
    }

    public decimal Price
    {
        get { return price; }
        set
        {
            if (value < 0) // грешка без else и без {} за 1 ред
                throw new ArgumentException("Цената не може да е отрицателна!");

            price = value;
        }
    }
}
```

Валидация в конструктор – тя е по-важна от валидацията в setter:

```
public class BankAccount
{
    private decimal balance;

    public decimal Balance // ВАЛИДАЦИЯ И В SETTER (допълнителна защита):
```

```

{
    get { return balance; }

    private set // само класът може да променя
    {
        if (value < 0)
            throw new InvalidOperationException("Балансът не може да стане отрицателен");

        balance = value;
    }
}

public BankAccount(decimal initialBalance) // ВАЛИДАЦИЯ В КОНСТРУКТОР
{
    if (initialBalance < 0)
        throw new ArgumentException("Балансът не може да е отрицателен при създаване!");

    balance = initialBalance;
}
}

```

Валидацията в конструктор се активира при създаването на нов обект неправилно, докато валидацията в setter – ако обектът е вече създаден (може и с празен конструктор) и после се опитаме да му придадем стойност, като достъпим свойството чрез точка . Затова е желателно да имаме валидации и на двете места – и в конструктора, и в setter, като особеното е, че в този случай често записването е обратното: написва се първо конструктора, после свойството (в обратен ред).

```

BankAccount b1 = new BankAccount(1234.567m); // нов обект ctor
BankAccount b2 = new BankAccount(-1m); // грешка заради ctor

```

```

BankAccount b2 = new BankAccount(765.432m); // нов обект ctor
b2.Balance = -987.654m; // грешка в setter

```

Статични членове **static без new** - класове и методи, които не създават нови обекти. Static променливите са общи за целия клас и принадлежат директно

на него. Примери са броячите Counter++, суми Total+= и е удачно да се поставят дори директно в конструктора. Достъпваме ги с `.` името на класа (.NET) – точката е много важна и вкарана в различни функционалности.

```
public class MathHelper
{
    public static double PI = 3.14159;
    public static int Add(int a, int b) => a + b;
}

double pi = MathHelper.PI;
int sum = MathHelper.Add(5, 3);
```

Какво са методите? Желателно ли е да имаме много логика в Main()? **Ctrl + .**

Масиви[] – еднотипни обекти с фиксирана дължина. Не можем да вмъкваме елементи в средата `.Insert()`, а при необходимост от увеличаване, използваме `.Resize()`.

Списъци<> - много по-гъвкави, по-бавни, можем спокойно да вмъкваме, добавяме и изтриваме елементи.

```
List<string> students = new List<string>();
students.Add("Мария");
students.Add("Петър");
```

Обхождаме с **foreach**, като при обхождане на списъци от родителски класове с наследници, за предпочитане е да използваме **var**, вместо да конкретизираме.

Използваме и много `.LINQ` вградени методи – четем подсказката и при **predicate** използваме `=>`

Речници

```
Dictionary<string, int> grades = new Dictionary<string, int>();
grades["Мария"] = 6;
grades["Петър"] = 5;
```

Интерфейси – пакети със задължителни методи и свойства (договор). Името им започва с I. Един клас може да наследява само един родител, но много интерфейси.

```
public interface IFlyable
{
    void Fly();
    int MaxAltitude { get; }
}

public class Bird : IFlyable
{
    public void Fly() => Console.WriteLine("Flapping wings");
    public int MaxAltitude => 1000;
}
```

Абстрактни класове – родителски класове, от които НЕ може да създаваме нови обекти. Те само дават ресурси на наследниците. Само наследниците създават нови обекти.

```
abstract class Animal {}
class Dog : Animal {}

Animal a1 = new Animal(); // греша
Dog d1 = new Dog();
```

Изключения

```
try
{
    int result = 10 / int.Parse(Console.ReadLine());
}
catch (DivideByZeroException ex)
{
    Console.WriteLine("Не може да делиш на нула!");
}
catch (FormatException)
{
}
```

```
    Console.WriteLine("Въведи число!");  
}  
finally  
{  
    Console.WriteLine("Това винаги се изпълнява");  
}
```

Най-интересните решения от срока

Любомир – задача за въвеждане на час и минути от потребителя и изчисляване на 15 минути по-късно:

```
int[] time = new int[2];  
time = Console.ReadLine().Split(':').Select(n => int.Parse(n)).ToArray();  
  
time[1] += 15;  
  
if (time[1] >= 60)  
{  
    time[1] -= 60;  
    time[0]++;  
}  
if (time[0] >= 24)  
{  
    time[0] = 0;  
}  
  
Console.WriteLine($"{time[0]:D2}:{time[1]:D2}");
```

Петър – задача за меню от зоомагазин:

```
Microsoft Visual Studio Debug Console

=== MENU ===
D) Dog food - 28.90lv
C) Cat food - 15.50lv
T) Toys - 12.75lv
A) Accessories - 8.30lv
Enter the letter for the item (or BILL if you are ready):
> c
You added C. Your total for now is: 15.50lv
> a
You added A. Your total for now is: 23.80lv
> r
> 1
> bill

===== BILL =====
Total: 23.80lv
Total in JPY: 1983.33JPY

THANK YOU! HAVE A NICE DAY!
Enter text: qwer
Reversed text: rewq

D:\Exercises\11 class\Dictionary_Peter\bin\Debug\net8.0\Di
Press any key to close this window . . .
```

Изпълнение:

```
Order.cs  Program.cs  Dictionary_Peter.Program

namespace Dictionary_Peter
{
    0 references
    internal class Program
    {
        0 references
        static void Main(string[] args)
        {
            създава обект
            Bill bill = new Bill();
            bill.ShowMenu();
            bill.ObrabotkaNaIzbori();  извиква методите
            bill.ShowSmetka();

            Console.WriteLine("\nTHANK YOU! HAVE A NICE DAY!");

            нов обект
            Reverse reverse = new Reverse();
            reverse.Obratno();  метод
        }
    }
}
```

В Main() само извиква методите. Използва речници, вместо switch()

Обърнете внимание, че `while(true) { if (условие за край) { break; } .... }`

```
public void ObrabotkaNaIzbori()
{
    while (true)
    {
        Console.Write("> ");
        string vhod = Console.ReadLine().ToUpper();

        if (vhod == "BILL")
            break;

        if (vhod.Length == 1 && menu.ContainsKey(vhod[0]))
        {
            char item = vhod[0];
            total += menu[item];
            Console.WriteLine($"You added {item}. Your total for now is: {total:F2}\n");
        }
    }
}
```

Класовете са в отделни файлове:

Петър Каригов

Няма оценка

Прегадени ([Преглед на историята](#))

<u>Order.cs</u> Текст		<u>Program.cs</u> Текст	
<u>Reverse.cs</u> Текст		<u>Screenshot 202...</u> Изображение	

Забележка за шльоковици.

Обръщане на текст – има много реализации

```
namespace IZPITVANE
{
    public class Reverse
    {
        public void Obratno()
        {
            Console.Write("Enter text: ");
            string input = Console.ReadLine();

            char[] letters = input.ToCharArray();

            Array.Reverse(letters);

            string reversed = new string(letters);
            Console.WriteLine("Reversed text: " + reversed);
        }
    }
}
```

Order.cs:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace IZPITWANE
{
    public class Bill
    {
        private Dictionary<char, decimal> menu;
```

```

        private decimal total;

        public Bill()
        {
            menu = new Dictionary<char, decimal>();
            total = 0.0m;
            InicializirajMenu();
        }

        private void InicializirajMenu()
        {
            menu.Add('D', 28.90m);
            menu.Add('C', 15.50m);
            menu.Add('T', 12.75m);
            menu.Add('A', 8.30m);
        }

        public void ShowMenu()
        {
            Console.WriteLine("=== MENU ===");
            Console.WriteLine("D) Dog food - 28.90lv");
            Console.WriteLine("C) Cat food - 15.50lv");
            Console.WriteLine("T) Toys - 12.75lv");
            Console.WriteLine("A) Accessories - 8.30lv");
            Console.WriteLine("Enter the letter for the item (or
BILL if you are ready):");
        }

        public void ObrabotkaNaIzbori()
        {
            while (true)
            {
                Console.Write("> ");
                string vrod = Console.ReadLine().ToUpper();

                if (vrod == "BILL")
                    break;

                if (vrod.Length == 1 &&
menu.ContainsKey(vrod[0]))
                {
                    char item = vrod[0];
                    total += menu[item];
                    Console.WriteLine($"You added {item}.
Your total for now is: {total:F2}lv");
                }
            }
        }

        public void ShowSmetka()
        {
            const decimal JPYRate = 0.012m;
            decimal totalJPY = total / JPYRate;

            Console.WriteLine("\n===== BILL =====");
            Console.WriteLine($"Total: {Math.Round(total, 2)}lv");

```

```
                Console.WriteLine($"Total in JPY: {Math.Round(totalJPY,
2)}JPY");
            }
        }
    }
}
```

Program.cs:

```
static void Main(string[] args)
{
    Bill bill = new Bill();
    bill.ShowMenu();
    bill.ObrabotkaNaIzbori();
    bill.ShowSmetka();

    Console.WriteLine("\nTHANK YOU! HAVE A NICE DAY!");

    Reverse reverse = new Reverse();
    reverse.Obratno();
}
```